

Multilayer Perceptron Trading Strategy on Proof of Stake Cryptocurrencies

Togay Atmaca(12268957), Jordan Cassella(12452926), Matthew Haimes(12452937)
, Kunj Shah(12258505), Yan Song Zhao(12448802)

March 2025

Abstract

This research explores the integration of on-chain data, staking data, and technical analysis as inputs into a Multi-Layer Perceptron (MLP) model to classify buy, hold, and sell signals on an hourly time frame. Pruned Exact Linear Time (PELT) segmentation algorithm is employed for change point detection, which detect structural shifts in price trends since early 2021. The MLP classifier incorporates on-chain data, staking data, and key technical indicators, including Exponential Moving Averages (EMA), Bollinger Bands, and Fibonacci Retracement levels as input, and it's designed to predict market movements across different market phases. The model is trained using the Adam optimizer, evaluated through standard performance metrics, and validated for robustness under repeated testing.

Keywords

Blockchain and Crypto-Related Terms

- Proof-of-Stake (PoS) – A blockchain consensus mechanism where validators (stakers) are selected based on the amount of cryptocurrency they hold and stake as collateral.
- On-Chain – Transactions recorded directly on a blockchain ledger, ensuring immutability and transparency.
- Blockchain – A decentralized, distributed ledger that records transactions securely across multiple nodes.
- Crypto – Digital or virtual currencies secured by cryptography, enabling decentralized transactions.
- Ledger – A digital record of transactions in a blockchain network.
- Gas Fees – Fees paid to miners or validators to process blockchain transactions.
- Staking – The process of locking up cryptocurrency to support network operations in exchange for rewards.

Machine Learning and Neural Networks

- Multilayer Perceptron (MLP) – A neural network with multiple layers of neurons (input, hidden, and output layers) used for classification and regression tasks.
- Neural Network – A computational model inspired by the human brain, consisting of interconnected layers of neurons for pattern recognition.

Time-Series and Statistical Analysis

- PELT (Pruned Exact Linear Time Algorithm) – A statistical method used for change point detection in time-series data.

1 Introduction

Cryptocurrency has evolved immensely over the past decade. Once considered speculative, it has now transformed into a recognized asset class for secure digital transactions. As the market expanded, so did the methodologies for creating and securing digital assets.

Bitcoin, the first cryptocurrency, was built on the Proof of Work (PoW) system. This required miners to solve complex mathematical problems to validate transactions, ensuring security through computational power. While PoW was secure, it was also highly energy-intensive. Circa 2020, Proof of Stake (PoS) emerged as a more energy-efficient alternative, and this method of network consensus has been the most popular. Instead of mining, PoS selects validators based on their staked cryptocurrency, reducing energy consumption while maintaining network security through economic incentives.

In our study, we utilize three types of features to predict price direction of various cryptocurrencies in a portfolio: on-chain data, staking data, and technical indicators. We utilize this feature set to predict one period ahead price movements using a 3-class MLP (Buy, Hold, Sell) for five cryptocurrencies; using the softmax activation function, position sizes are computed as a percentage of prediction probabilities in a dynamic portfolio.

In Section 2, we further explain the details of the data we’re using for the experiment. In Section 3, we outline the implementation of the algorithm, as well as the explanation of the PELT Algorithm used for change in point detection normalization in Section 3.1. For the rest of Section 3, we also introduce the computation of technical indicators and methodology for signal (label) generation, explain our train-test-split structure and data transformations, and introduce the Neural Network Architecture as well as show how we evaluate the model and the respective performance metrics. In Section 4, we explain the portfolio allocation methodology. In Section 5, we include an Analysis for each coin and on the portfolio. In Section 6, we include a conclusion where we also provide suggested future applied research.

2 Data

This study analyzes the following cryptocurrencies:

- Avalanche (AVAX)
- Polygon (MATIC)
- Solana (SOL)
- Near (NEAR)
- Polkadot (DOT)

Hourly pricing data for these assets was sourced from Binance via the CoinAPI. On-chain analytics were obtained from Dune API, while staking data was collected from StakingRewards. These cryptocurrencies were selected based on their high market capitalization and availability of historical data from at least 2021 to 2025.

On-chain blockchain data contains the information that is recorded directly on the blockchain ledger, which includes details such as transaction volumes, the number of active wallet addresses, token transfers, and processing difficulty, particularly for Proof of Work coins. However, our analysis is on Proof of Stake coins, where we primarily have gas fees and transaction size per block as our main features. Since all transactions on a blockchain are publicly accessible, this data provides a transparent and verifiable record of network activity. For example, high transaction volumes and increasing active addresses can signal growing user interest and network adoption, potentially leading to price appreciation. Additionally, shifts in token distribution or movements into cold storage and staking contracts can reduce the circulating supply, creating upward pressure on prices. Therefore, on-chain data provide insight into network health and stability, which

can enhance investor confidence and influence price direction.

Staking data is a key component of our analysis, as it includes blockchain metrics such as the staking ratio, total staked coins, total staking wallets, real reward rate, annualized rewards in USD, total active validators. These metrics provide valuable insights into the supply dynamics, network security, and network flow of the blockchain for the selected cryptocurrencies. For example, percentage of eligible tokens that are currently being staked or delegated to the network. This metric can provide insights into the level of network participation among token holders, the health and security of the network, and how committed the community of token holders are. Moreover, robust staking activity is a strong signal of point in time investor confidence and commitment to the network; we take advantage of this relationship by integrating staking data into our study. As a result we obtain a more comprehensive view of market sentiment and economic incentives, complementing the pricing and on-chain analytics used in our model.

The model undergoes feature engineering and normalization to handle high volatility. The PELT algorithm is used for change point detection in price movements.

3 Implementation of Algorithm

This section walks through the steps of the trading strategy. The trading strategy follows these steps:

1. Collect and preprocess data, normalizing on-chain and technical indicator values.
2. Apply PELT segmentation to detect regime changes.
3. Train an MLP classifier using labeled buy, hold, and sell signals.
4. Evaluate performance using backtesting on hourly data.

3.1 Data Splitting, Scaling, and Balancing

The data is split into 85% training and 15% testing sets, preserving the temporal order to maintain statistical validity.

Features are scaled using `StandardScaler`, which is fit on the training data and applied to both training and testing sets. This sets the mean to 0, and variance to 1.

Class weights are computed to balance the dataset, preventing the model from overfitting to the dominant `hold` class. This is then used as a part of the model fitting process.

3.2 Pruned Exact Linear Time (PELT) Algorithm

The PELT algorithm is used for change point detection in the price movements of the cryptocurrencies.

3.2.1 Problem Formulation

Given a cryptocurrency time series $\mathbf{X} = \{X_1, X_2, \dots, X_T\}$, we seek to identify change points $\{t_1, t_2, \dots, t_K\}$ that minimize:

$$\min_{\mathbf{t}} \sum_{i=0}^K C(X_{t_i+1:t_{i+1}}) + K\beta \quad (1)$$

where $C(X_{t_i+1:t_{i+1}})$ is the segment cost, and β controls complexity.

3.2.2 Cost Function (RBF Kernel)

The segment cost using an RBF kernel is:

$$C(X_{a:b}) = \sum_{i=a}^b k(X_i, X_i) - \frac{1}{(b-a)} \sum_{i=a}^b \sum_{j=a}^b k(X_i, X_j) \quad (2)$$

where $k(X_i, X_j) = e^{-\gamma \|X_i - X_j\|^2}$.

3.2.3 Optimization (PELT Algorithm)

Using dynamic programming, the optimal segmentation satisfies:

$$F(t) = \min_{\tau < t} [F(\tau) + C(X_{\tau:t}) + \beta] \quad (3)$$

3.2.4 Pruning for Efficiency

PELT eliminates unnecessary change points using:

$$C(X_{s:e}) + C(X_{e:t}) + \beta \geq C(X_{s:t}) + \beta \quad (4)$$

This reduces complexity from $O(T^2)$ to $O(T)$, making segmentation feasible for large datasets.

3.2.5 Effectiveness in Model

The PELT algorithm is relevant for analyzing cryptocurrency time series data due to the high volatility surrounding this asset class. PELT recognizes drastic changes caused by factors such as market news and sudden liquidity shifts. By applying PELT, we can remove levels of random noise and help avoid overfitting in predictive models. Additionally, the algorithm allows for effective normalization of price data, improving the robustness and accuracy of statistical and machine learning models in crypto market analysis.

To ensure stationarity within each segmented regime, we normalize price data using:

$$Z_t = \frac{X_t - \mu_{\text{segment}}}{\sigma_{\text{segment}}} \quad (5)$$

where:

- X_t is the raw price at time t .
- μ_{segment} is the mean price within the detected segment.
- σ_{segment} is the standard deviation of prices in the segment.

This transformation ensures that each segment has a standardized distribution, allowing models to learn meaningful patterns without being affected by varying volatility levels.

3.3 Technical Indicators

The technical indicators are computed using a value $n = 24$ for the lookback. This value is used for multiple indicators.

The Bollinger Band (BB) computes a simple moving average μ_n and a standard deviation σ_n , both on a rolling basis with lookback n , and the value of the indicator is

$$BB_t := \frac{P_t - \mu_{n,t}}{\sigma_{n,t}}$$

The Exponential Weighted Moving Average (*EMA*) is done by computing two EMAs for short and long lookbacks set at $n_1 = 20$, $n_2 = 50$, and the value of the indicator is the difference of the two, defined by

$$EMA_t := EMA_{n_1,t} - EMA_{n_2,t}$$

The Relative Strength Index (*RSI*) requires two intermediate variables U , D . The up move variable U takes the simple 1-period difference over the past n periods, and clips all negative values to 0. The down move variable D takes the simple 1-period difference over the past n periods, and clips all positive values to 0. Then, the *RSI* takes the ratio

$$RSI_t := \frac{U_{t,n}}{U_{t,n} + D_{t,n}}$$

Unlike the usual definition of RSI which takes on the range $[0, 100]$, this definition takes on the range $[0, 1]$.

The Hour of Day (H), the Day of Week (A), and the Month of Year (m) are integer-valued metrics. The ranges are $1 \leq H \leq 24$, $1 \leq A \leq 7$, $1 \leq m \leq 12$.

3.4 Signal Generation

We compute the price difference, which is the arithmetic difference between consecutive closing prices. Then, a threshold is set at half the standard deviation of the price difference. Based on the magnitude of this threshold, the signal is categorized into:

- **Buy:** When the price difference is higher than the positive threshold.
- **Sell:** When the price difference is lower than the negative threshold.
- **Hold:** When the price difference is within the positive and negative threshold.

3.5 Neural Network Architecture

A multilayer perceptron is a type of artificial neural network consisting of multiple layers of neurons. The input layer receives the raw inputs from the columns of the data. The hidden layers process and learn complex patterns in the data by applying an activation function to a weighted sum of its inputs. The output layer produces the final probability prediction for **buy**, **sell**, and **hold**. Each unit within one layer will produce an output, which will be the input to every unit of the next layer. The data will pass through the MLP layer by layer, while being transformed at every unit, until it reaches the final output layer.

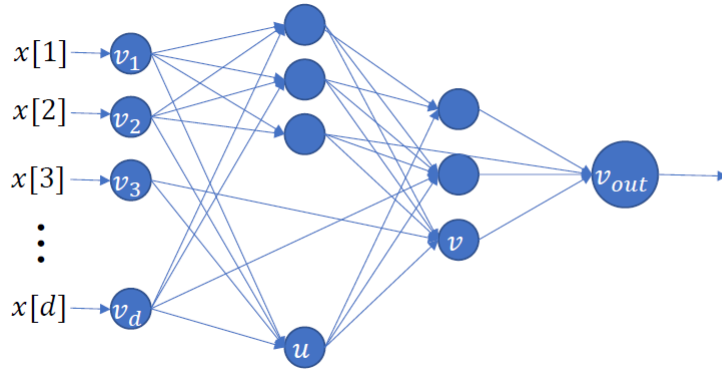


Figure 1: Diagram of a MLP architecture

For this particular neural network, the architecture has an input layer matching the number of features. There are two hidden dense layers with 64 and 32 neurons, using **LeakyReLU** activation. Finally, there is a single node in the output layer, using **softmax** activation.

The **LeakyReLU** activation function is similar to the standard ReLU activation function, except a small positive gradient is set when the unit is inactive. If the gradient is 0 like the inactive ReLU unit, the learning process will stop altogether at this unit, which prevents the model from reaching the optimal solution. This is called the vanishing gradient problem, which is exactly what **LeakyReLU** mitigates.

ReLU Vs Leaky ReLU

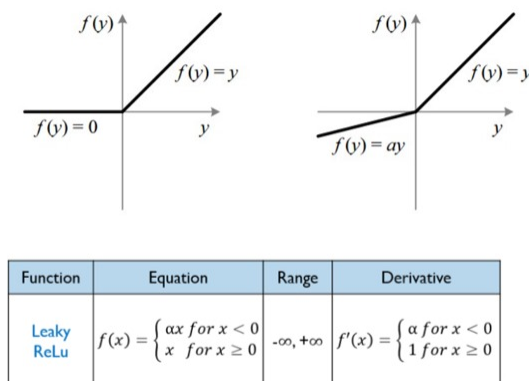


figure 2: ReLU vs LeakyReLU

The **softmax** activation function transforms the raw inputs into a vector of probabilities, by taking into account the relative magnitudes of each of the inputs. This is how the model expresses its belief for the probabilities of the label that it's trying to predict.

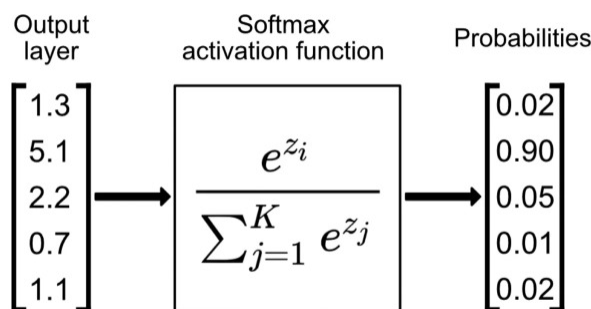


figure 3: Softmax Activation Function

The model is trained using the **Adam** optimizer, and trained for 50 epochs with a batch size of 32.

3.6 Evaluation and Performance Metrics

Model prediction accuracy is evaluated on both training and testing sets. A confusion matrix (visualized using Seaborn) and a classification report help assess misclassifications among the Buy, Sell, and Hold signals.

The starting capital is set to \$100,000, to avoid any issues with capital constraints. The backtest assumes that each buy or sell signal initiates a position in that token. The sizing of this position is determined via the strategy described in section 4.

The cumulative returns, total profit-and-loss (PnL), and annualized Sharpe ratio are computed to assess trading performance.

To test for model robustness, the model is retrained 10 times with the same architecture to assess stability. The performance metrics are recorded for each run, and we report them to demonstrate the robustness of the model.

4 Portfolio Allocation

4.1 Position Sizing

Our portfolio allocation strategy takes advantage of the probabilities output by the softmax activation function. At the last layer of the MLP network, the softmax activation function returns both a classification prediction and associated probabilities for each of the possible classifications in the output space. For our strategy we utilized a set initial capital of 100,000 USD with no leverage. For transaction cost we used 50 bps per trade.

In setting up the portfolio, we assume the equilibrium assumption and assign each token 20% of the initial starting capital as its investment pool. When we receive a 'BUY' signal, we take the corresponding softmax probability associated with the 'BUY' classification and treat it as a strength indicator. We size the position by taking the strength indicator divided by some constant C to get some proportion p of our maximum capital allocation which is used for the position sizing. This approach allows us to add varying degrees of confidence to any given position, ensuring that our model doesn't take identical actions on signals of varying strength.

We perform a symmetrical procedure when we receive a 'SELL' signal. Our team experimented with an additional parameter for asymmetric sizing between 'BUY' and 'SELL' signals, which in essence would have replaced our C parameter with C_B, C_S parameters. However, we found this extra layer of complexity diluted the focus of the model and it's ability to predict directional movements. Our team reworked the asymmetry between 'BUY' and 'SELL' signals in our implementation of the stoploss parameter, which triggers more excessive selling in times of portfolio stress.

'HOLD' signals lead to no changes in the portfolio holdings.

4.2 Risk Control

A portion of our risk control is a result of our equilibrium assumption made in our position sizing. In equalizing our maximum allocation to any given token, we introduce no Bayesian priors as to which token is expected to perform best. Furthermore, we protect our portfolio from overexposure to any given token should one experience a 'hot streak' (generating consecutive 'BUY' signals before experiencing a crash to normal price levels).

Our risk control strategy also considers the importance of asymmetrical position actions. In an instance such as the one described above, our strategy realizes the importance of closing out a position entirely. This risk is controlled via our stoploss parameter, which is set to trigger when a token's portfolio value drops by FILL IN PERCENT% in a given time interval.

5 Analysis



Figure 1: Comparison of coins versus Fama-French factors

As seen in Figure 1, the backtested strategy appears to outperform the Fama-French market factors, particularly after mid-2024. Additionally, the regression's very low R^2 of less than 1% and insignificant beta coefficients (much lower than 1) for all coins indicate that these factors do not meaningfully explain the strategy's excess returns; furthermore, exposure to risk premia according to the Fama-French framework don't seem to explain the returns of our strategy.

According to Figure 2a, we can see that the MLP strategy run on AVAX delivers strong risk-adjusted returns, since we observe a 2.18 Annualized Sharpe ratio. Additionally, we observe negative correlation to BTC and negative downside beta, meaning our strategy delivers positive returns when BTC has a drawdown. We can view BTC as a broad market factor for cryptocurrencies, as many coins are highly correlated to it and have substantial beta estimates to BTC.

According to Figure 2b, we can see that the MLP strategy on MATIC produces strong risk-adjusted returns, since we observe a 1.87 Sharpe ratio. Similar to AVAX, MATIC has negative correlation and negative downside beta to BTC. Although annualized returns and Sharpe ratio are similar for AVAX and MATIC, the graphs in Figure 1 show very different dynamics.

According to Figure 2c, we can see that the MLP strategy on MATIC produces a 1.06 annualized Sharpe ratio and 13% returns. Here, we have a higher correlation to BTC at 0.34 and a positive downside beta to BTC at 0.06. Both of these values, although not as desirable characteristics as AVAX and MATIC, are still relatively low values for these metrics. Even with a 0.34 correlation to BTC, we still view this figure as favorable to the robustness of the strategy.

According to Figure 2d, we can see that the MLP strategy on DOT has an annualized Sharpe ratio of 0.31 and returns of 2%, which is much lower than the other coins. However, DOT has a downside beta of 0.05, which is still very low even if it's not negative. Furthermore, we observe a very low annualized volatility of returns, which can still be quite useful in lowering the overall volatility of returns of a portfolio of crypto assets.

According to Figure 2e, we see that the MLP strategy on SOL has an annualized Sharpe ratio of 1.73 and returns of 45%. However, our downside beta and correlation to BTC are much higher, taking values of 0.20 and 0.50, respectively. Therefore, we can say the dynamics of the overall crypto market does explain more of our strategy's returns on SOL, compared to the other coins that we utilized.

According to Figure 3, we can see that we have positive ending PnL for all coins in the portfolio except for MATIC. We can see that at large, our portfolio management methodology is quite robust, as it functioned as intended. Furthermore, we were able to effectively aggregate signals from multiple assets into a dynamic portfolio that isn't self-financing.

Performance Metrics	
mean_return_annual	0.35
vol_annual	0.16
sharpe_ratio	2.15
sortino_ratio	2.55
max_drawdown	-0.07
calmar_ratio	4.99
VaR_95	0.00
CVaR_95	0.00
beta (to BTC)	0.10
information_ratio (to BTC)	-1.01
correlation_with_BTC	0.31
upside_capture_ratio (to BTC)	0.09
downside_capture_ratio (to BTC)	0.07
downside_beta (to BTC)	0.10

(a) AVAX, Metrics

Performance Metrics	
mean_return_annual	0.19
vol_annual	0.10
sharpe_ratio	1.87
sortino_ratio	2.28
max_drawdown	-0.06
calmar_ratio	3.18
VaR_95	0.00
CVaR_95	0.00
beta (to BTC)	-0.07
information_ratio (to BTC)	0.17
correlation_with_BTC	-0.36
upside_capture_ratio (to BTC)	-0.07
downside_capture_ratio (to BTC)	-0.08
downside_beta (to BTC)	-0.08

(b) MATIC, Metrics

Performance Metrics	
mean_return_annual	0.13
vol_annual	0.12
sharpe_ratio	1.06
sortino_ratio	1.40
max_drawdown	-0.07
calmar_ratio	2.03
VaR_95	0.00
CVaR_95	0.00
beta (to BTC)	0.08
information_ratio (to BTC)	-1.27
correlation_with_BTC	0.34
upside_capture_ratio (to BTC)	0.10
downside_capture_ratio (to BTC)	0.09
downside_beta (to BTC)	0.06

(c) NEAR, Metrics

Performance Metrics	
mean_return_annual	0.02
vol_annual	0.06
sharpe_ratio	0.31
sortino_ratio	0.38
max_drawdown	-0.03
calmar_ratio	0.60
VaR_95	0.00
CVaR_95	0.00
beta (to BTC)	0.04
information_ratio (to BTC)	-1.39
correlation_with_BTC	0.34
upside_capture_ratio (to BTC)	0.02
downside_capture_ratio (to BTC)	0.02
downside_beta (to BTC)	0.05

(d) DOT, Metrics

Performance Metrics	
mean_return_annual	0.45
vol_annual	0.26
sharpe_ratio	1.73
sortino_ratio	2.11
max_drawdown	-0.10
calmar_ratio	4.51
VaR_95	0.00
CVaR_95	0.01
beta (to BTC)	0.26
information_ratio (to BTC)	-2.06
correlation_with_BTC	0.50
upside_capture_ratio (to BTC)	0.27
downside_capture_ratio (to BTC)	0.26
downside_beta (to BTC)	0.20

(e) SOL, Metrics

Figure 2: Comparison of coins versus Fama-French factors



Figure 3: Portfolio End PnL

Performance Metrics	
mean_return_annual	0.08
vol_annual	0.26
sharpe_ratio	0.31
sortino_ratio	0.36
max_drawdown	-0.22
calmar_ratio	0.35
VaR_95	0.00
CVaR_95	0.01

Figure 4: Portfolio Metrics

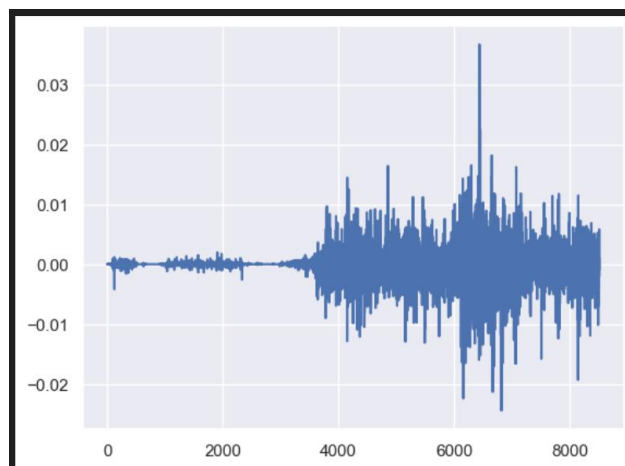


Figure 5: Portfolio Overview

According to Figure 4, we can see that the annualized mean returns are 8% and the Sharpe ratio is 31%. Additionally, our max drawdown for our final portfolio is -0.22.

6 Conclusion

From this model, we demonstrated that on-chain data is valuable for designing trading strategies in cryptocurrencies. We also demonstrated that MLP with LeakyReLU activations is effective for learning the complexities in the on-chain and technical data.

We were able to train the MLP model in under 2 minutes for all the coins in our analysis, for approximately 4 years of hourly data. This shows that the model can be easily ported to a different asset, or for a different time scale.

While there are difference in the training accuracy for different coins due to the inherent idiosyncrasies of each individual coin, the training and testing accuracy for the same coin is highly stable. This suggests that the MLP model itself is fairly robust from the stochastic optimization process. Additionally, the prediction accuracy between the training and the test sets have been reasonably close, suggesting that the model is not overfit.

Finally, there are many promising ways to improve the model, many of which can dramatically improve the expressive power of the MLP model. This can potentially lead to significant improvements in strategy performance.

7 Drawbacks

Throughout the development of this strategy, we encountered several challenges that impacted data collection, model accuracy, and overall predictive performance. Accessing reliable and cost-effective on-chain and staking data for various cryptocurrencies proved to be a significant hurdle, requiring us to leverage multiple third-party APIs such as DuneAPI, COINAPI, and StakingRewards. However, differences in data structures across cryptocurrencies made it difficult to establish consistent patterns for prediction. Additionally, gaps and inconsistencies in certain datasets, such as NEAR and DOT, where missing values were replaced with zeros, introduced errors that affected the model’s performance. These issues highlighted the complexity of analyzing fundamentally different cryptocurrencies under a unified framework. Moving forward, we aim to expand our dataset to include a broader universe of assets over a consistent time scale and explore deeper relationships between PoS and PoW mechanisms to develop a more robust MLP model.

Since our strategy encompassed building an MLP model for each coin, we don’t reap the benefits of stacking, which is the idea that we can train one model on all the data (using dummy variables to denote the particular coin from the sample), which allows us to substantially increase the number of samples our model can train with.

A deeper and wider MLP model will be able to capture more complexities in the structure of the data. However, we chose not to explore architectures that were much bigger due to the significantly increased model training time. However, with more computational resources, it’s possible that a larger model can outperform our implementation.

The signal itself is labeled into three classes, **buy**, **sell**, **hold**. The model could only express its belief about the probability between these three classes, but it cannot express any beliefs about the magnitude of the predicted move. Moving away from a classification paradigm into a regression paradigm will give the model even more flexibility.

If the trading strategy is to be implemented for real time trading, adding more data points can cause the PELT segments to change retroactively under extreme market conditions. The model is designed to perform under data that is normalized using these segments, but we do not have access to regime changes on a walk-forward basis, and these regime changes may cause the model to struggle.

Also, an interesting edit to the structure of our train-test-split would be a 90% train, 10% test since we’ll have more data to train with; due to our hourly data frequency, we’ll have a sufficient amount to perform an out-of-sample backtest.

Another drawback is that we did not use leverage to fully exploit the strategy. With marginal funding costs, this strategy may have potential for greater profits.

8 Citation

- Kim, G., Shin, D.-H., Choi, J. G., & Lim, S. (2022). A deep learning-based cryptocurrency price prediction model that uses on-chain data. *IEEE Access*, 10, 56232–56247.
<https://doi.org/10.1109/ACCESS.2022.3177888>
- Parente, M., Rizzuti, L., & Trerotola, M. (2024). A profitable trading algorithm for cryptocurrencies using a neural network model. *Expert Systems with Applications*, 238, 121806.
<https://doi.org/10.1016/j.eswa.2023.121806>

9 Acknowledgments

- We would like to extend our sincere gratitude to the team at Robinhood for hosting the trivia event that our group had the pleasure of participating in—and winning. The prize winnings played a crucial role in funding the acquisition of the data we used in our project. Your innovative event not only provided a fun and engaging experience but also directly supported our research efforts, and for that, we are truly thankful.
- We would also like to express our heartfelt thanks to the Financial Mathematics program for generously helping fund the acquisition of the data used in our project. Your support not only provided us with the necessary resources to advance our research but also affirmed the value of collaborative efforts.